ADOPTION

Champion kit

A playbook for engineers advocating Claude Code internally: what to share, how to answer questions, and how to grow adoption on your team.

This page is for individual engineers who are already using Claude Code and want to help their team adopt it. It covers what to share, how to answer the questions you will get, a thirty-day playbook, and responses to common concerns.

Adoption of a developer tool rarely happens because of a rollout announcement. It happens because someone on the team begins using the tool well, talks about it openly, and makes it easy for others to follow. The work you do as a champion has a disproportionate effect: every example you share shortens the learning curve for the engineers who come after you, and every question you answer in public turns one person's experience into something the whole team can build on. You are acting as a multiplier for your team, not a help desk, and this guide is structured to keep the role sustainable on those terms.

The champion role

The role consists of three behaviors that reinforce one another.

Behavior	What it looks like in practice	Why it matters
Share what you discover	Post the prompts, screenshots, and small wins from your own work in the places your team already reads, such as an engineering channel, a standup thread, or a pull-request description.	Examples drawn from your own codebase are more persuasive than any external documentation, because colleagues can see exactly how the tool applies to the problems they share with you.
Be the person people ask	When a colleague asks how you accomplished something, respond with the actual prompt you used so they can apply it directly to their own task.	A concrete, runnable example removes the gap between curiosity and a first successful use, which is where most adoption efforts stall.
Grow the circle	Establish a small number of lightweight, recurring habits, such as a dedicated channel or a weekly thread, so that momentum continues even when your	Adoption that depends on a single person is fragile. Adoption that is carried by shared habits continues to compound on its own.

Behavior	What it looks like in practice	Why it matters
	attention is elsewhere.	

Most of this fits naturally inside the work you are already doing. The difference is a small amount of additional intention about where your discoveries are posted and how your answers travel.

What this should cost you

Set expectations with yourself and with your lead. The activities below are intended to fit inside a normal working week, and the role should remain a multiplier on your existing work rather than an additional support responsibility.

Activity	Time per week	Guidance
Posting wins and prompts	About 15 minutes	Capture these in the moment with a screenshot and one or two sentences; avoid turning them into formal write-ups.
Answering questions in a shared channel	About 20 minutes	Answer publicly once, then link back to that answer when the question recurs.
Hosting a weekly show-and-tell thread	About 5 minutes	You post the opening prompt; the team supplies the content.
Optional pairing or walkthroughs	0 to 30 minutes	Reserve this for colleagues who are genuinely blocked, and offer the **Quickstart** link before scheduling time.

Share what you discover

Your own experience is the most persuasive material your colleagues will encounter, because it is specific to the codebase, workflows, and problems you all share. Documentation tells people what is possible; your posts show them what is actually working in your environment.

What is worth sharing

The most useful posts describe a technique a colleague can reuse tomorrow rather than an outcome that is already complete. Techniques compound as they spread through a team; status updates do not.

Examples of reusable techniques:

- “I learned that @-mentioning a directory works. Pointing it at `@src/components/` and asking

which were missing tests surfaced two I had overlooked."

- "Plan mode (`Shift+Tab`) shows exactly which files will be touched before any edit is made, which is why I am comfortable using it on shared code."
- "I configured a Stop hook so I receive a desktop notification when a long task completes. Configuration is in the thread."
- "Running `/init` generates a `CLAUDE.md` from the repository so the assistant stops re-asking about our conventions."

Where to share it

Post wherever your team already reads. The goal is to place examples in the path of normal work rather than to create a destination.

Location	Best suited for	Recommended format
A `#claude-code` or general engineering channel	Discoveries, prompts, and "today I learned" moments	A screenshot accompanied by one or two sentences of context
Pull-request descriptions	Demonstrating the approach on real code that reviewers are already reading	A single line such as "Claude and I did this refactor; happy to walk through the approach."
Standups or weekly written updates	Normalizing usage with leads and skip-level managers	One sentence describing one concrete outcome
Team wiki or internal documentation	Durable patterns, custom skills, and `CLAUDE.md` examples	A short page, linked from the channel topic so it remains discoverable

The format that works

A screenshot accompanied by a single line of context, or a brief before-and-after description, is generally the right level of detail. Keep each post short enough that someone scrolling past still absorbs the point. A long write-up tends to be saved for later and forgotten, whereas a short post with a screenshot tends to be copied and tried.

The example posts below illustrate tone and length; adapt them rather than copying verbatim.

```
Learned today that @-mentioning a directory works. I pointed it
at
@src/components/ and asked which components were missing tests,
and it
surfaced two I had forgotten about.
```

```
I configured a Stop hook so I receive a desktop notification when
a long
task completes. I started a refactor, stepped away, and was
notified when
it finished. Configuration is in the thread.
```

```
Plan mode is the reason I am comfortable using this on code that
matters.
Press Shift+Tab until you see "plan"; it lays out exactly which
files it
intends to touch before changing anything.
```

Be the person people ask

Once you have shared a few examples, questions will follow. This is where the champion role has the greatest leverage, because a good answer to one person frequently unblocks several others who are watching the same channel.

Answer with a prompt rather than an explanation

When a colleague asks how you accomplished something, the most useful response is the prompt you actually used. They will learn more from running that prompt against their own problem than from any description you could write, and it gives them something they can act on immediately.

```
Colleague: How did you get it to find that race condition?

Champion: I asked, "The test in @tests/scheduler.test.ts is
flaky, figure
out why," and it traced two unjoined promises in the scheduler.
Try the
same phrasing on your test.
```

Point at the feature rather than the documentation

A response such as “Try plan mode, press `Shift+Tab` until you see it” is more useful in the moment than a link to the documentation. If the person needs more depth later they will find it on their own; right now they need the single thing that unblocks them.

Questions you are likely to hear

Question	Suggested response	Follow-up resource
”What should I try it on first?”	Recommend a real but contained task, ideally a bug or chore the person has been postponing because it is tedious rather than difficult.	**Common workflows**
”How do I trust it with my code?”	Introduce plan mode: pressing `Shift+Tab` cycles into it, Claude proposes exactly what it intends to change, and nothing is modified until the user approves.	**Permissions**
”Is the setup worth the effort?”	Installation takes roughly two minutes, runs in the terminal, and requires no IDE extension. Running `/init` once is sufficient to begin working.	**Quickstart**
”It produced an incorrect result.”	Encourage them to provide the failure back to Claude. Pasting the error message or failing test is far more effective than rephrasing the original request.	**Common workflows**
”It does not understand our codebase conventions.”	Suggest running `/init` to generate a `CLAUDE.md` file, then adding the team’s conventions, test commands, and any directories that should be avoided.	**Memory**
”Is this just autocomplete?”	Offer a brief demonstration in which Claude explains an unfamiliar file, traces a bug across services, or drafts a migration plan. These tasks require reasoning across the repository rather than completing a single line.	A two-minute live demonstration
”What about security and data handling?”	Refer this question to your administrator. Your organization’s deployment and data-handling policy is already configured, and champions should not improvise this answer.	**Security** · **Data usage**

Grow the circle

The objective is not to build a program or to own a rollout. It is to establish a small number of lightweight habits that allow momentum to continue after you have stopped actively driving it. When questions in the channel are being answered by people other than you, the role has done its job.

Patterns that tend to work

Pattern	How to run it	Effort required
A dedicated channel	Create a `#claude-code` channel (or a recurring thread in an existing one), pin the **Quickstart** link and one strong example, and answer questions publicly so each answer benefits everyone watching.	About five minutes to set up, then ambient
A weekly show-and-tell thread	Each Friday, post "What did Claude help you with this week?" No preparation, slides, or meeting are required; screenshots and short descriptions are sufficient.	About two minutes per week
Share a custom skill	Post your most useful `.claude/skills/<name>/SKILL.md` file, for example a `/ship` skill that runs tests and lint before committing, with a one-line description. Because skills are plain Markdown, colleagues can adopt them immediately.	About five minutes per skill
Generate a setup guide from your own usage	Run `/team-onboarding` in a project you have spent real time in. Claude scans your recent sessions, commands, and MCP servers, then produces a guide a new teammate can paste as their first message to replay your setup. Pin it in the channel.	About two minutes
Pair on a first task	Offer a single fifteen-minute pairing session to anyone getting started. One successful outcome on their own code is more persuasive than any presentation.	About fifteen minutes per person
Identify the next champion	The colleague who asks you the most questions is usually ready to take on this role. Forward them this page and divide the channel responsibilities between you.	Negligible

Thirty-day playbook

If a loose plan is helpful, the sequence below reflects what tends to work across most teams. Adjust freely to fit your context.

1 Week 1: Seed the channel

Create the channel, pin the **Quickstart**, and post two or three of your own examples with the prompts included.

Signal that it is working: a few colleagues react or reply, and at least one question is asked in the channel.

2 **Week 2: Start the rhythm**

Start the weekly show-and-tell thread, answer every question publicly, and share one custom skill or `CLAUDE.md` snippet.

Signal that it is working: someone other than you posts an example of their own.

3 **Week 3: Pair and consolidate**

Offer two or three short pairing sessions and consolidate the most common questions and answers into a pinned FAQ message.

Signal that it is working: you see repeat usage, with the same colleagues returning rather than trying once and stopping.

4 **Week 4: Hand off**

Identify a second champion and share a brief summary of what is working and what is not with your lead or administrator.

Signal that it is working: questions in the channel are being answered by people other than you.

When someone wants to go deeper

You are the warm introduction rather than the onboarding program. When a colleague moves past "should I try this" into "how do I become effective with it," point them to the **Quickstart** and **Common workflows** pages. They contain short sections covering the features that are genuinely useful but difficult to discover on your own.

Respond to common concerns

Healthy skepticism is expected; engineers should be cautious about tools that touch their code. The most effective response is rarely to argue the general case. Instead, acknowledge the concern, offer a brief reframe, and propose one concrete demonstration on the person's own code. Most concerns are resolved by a single successful experience.

Concern	Suggested response	Evidence to offer
”I am faster without it.”	That is likely true for code the person writes routinely. Suggest trying it on the work they tend to avoid: legacy files, unfamiliar services, or test scaffolding, where the leverage is highest.	Time one tedious task both ways and compare.
”I do not trust AI to touch production code.”	Agree that no change should land unread. Plan mode combined with normal diff review means nothing is applied that the engineer has not inspected, the same standard as any pull request.	Demonstrate plan mode on a real file.
”It will make junior engineers weaker.”	Used well, it is an effective explainer. Encourage junior engineers to ask Claude to explain a file and its call sites before asking it to change anything.	Run “Explain @file and where it is called from” together.
”I tried it once and it hallucinated.”	This is usually a context problem rather than a model problem. @-mentioning the relevant files, running `/init`, and providing the actual error output typically resolves it.	Re-run their original prompt with proper `@`-context.
”We do not have time to learn another tool.”	Claude Code is a terminal command rather than a platform. If it does not return value within the first session, it is reasonable to set it aside.	A two-minute install followed by one real bug.

Quick-reference sheet

The techniques below are the ones that most reliably move someone from a first trial to daily use. Pin this table in a channel or share it on its own.

Technique	How to apply it
Provide the right context	Use `@file` or `@directory/` references, or paste the error or log output directly. Supplying relevant context is more effective than elaborate prompting.
Review the plan before the edit	Press `Shift+Tab` to enter plan mode. Claude will describe the intended changes for your approval before executing them.
Teach it your repository	Run `/init` to generate a `CLAUDE.md` file, then add your conventions, test commands, and any directories that should not be modified. See **Memory**.
Reuse a workflow	Save a `SKILL.md` file in `.claude/skills/<name>/` to create a `/name` skill that the entire team can use. See **Skills**.
Stay informed during long tasks	Configure a Stop hook to receive a desktop notification when a long-running task completes. See **Hooks**.
Recover from an incorrect result	Rather than rephrasing the request, paste the failing test or stack trace back to Claude and ask it to address that specific failure.

Technique	How to apply it
Keep edits surgical	Ask for a diff, or specify “only change X.” Claude respects scope when scope is stated.

Claude Code is updated frequently. Verify version-specific details against the **documentation home page** before distributing this material internally.